

Name: Kenny Tang

Student ID: 2802517733

1. Compare the results for two versions of the implementations (own and thread library).

Both versions produce the same output and behavior. The difference is only at the primitive level: the own semaphore manually blocks threads via a condition variable while the counter is 0, whereas `sem\_t` does this inside the kernel (futex-based on Linux). Under low contention like this lab, the difference is negligible.

2. Use *time* shell program to output the running times for each version of your program.

```
→ lab-5 git:(main) X make time
gcc -Wall -Wextra -g -pthread -o own_sem own_sem.c
gcc -Wall -Wextra -g -pthread -o std_sem std_sem.c

>>> Timing OWN SEMAPHORE version...

time ./own_sem
=== OWN SEMAPHORE ===
[OWN] Writer-1 | write #1 | 20:52:50.573
[OWN] Writer-2 | write #1 | 20:52:50.573
[OWN] Reader-2 | read #1 | 20:52:50.623 | "Writer-2 | write #1 |
20:52:50.573"
[OWN] Reader-1 | read #1 | 20:52:50.623 | "Writer-2 | write #1 |
20:52:50.573"
[OWN] Writer-1 | write #2 | 20:52:50.723
[OWN] Writer-2 | write #2 | 20:52:50.723
[OWN] Reader-2 | read #2 | 20:52:50.724 | "Writer-2 | write #2 |
20:52:50.723"
[OWN] Reader-1 | read #2 | 20:52:50.724 | "Writer-2 | write #2 |
20:52:50.723"
[OWN] Reader-2 | read #3 | 20:52:50.824 | "Writer-2 | write #2 |
20:52:50.723"
[OWN] Reader-1 | read #3 | 20:52:50.824 | "Writer-2 | write #2 |
```

```

20:52:50.723"
[OWN] Writer-1 | write #3 | 20:52:50.873
[OWN] Writer-2 | write #3 | 20:52:50.874
[OWN] Reader-2 | read #4 | 20:52:50.924 | "Writer-2 | write #3 |
20:52:50.874"
[OWN] Reader-1 | read #4 | 20:52:50.924 | "Writer-2 | write #3 |
20:52:50.874"
[OWN] Writer-1 | write #4 | 20:52:51.024
[OWN] Writer-2 | write #4 | 20:52:51.024
[OWN] Reader-1 | read #5 | 20:52:51.024 | "Writer-2 | write #4 |
20:52:51.024"
[OWN] Reader-2 | read #5 | 20:52:51.024 | "Writer-2 | write #4 |
20:52:51.024"
[OWN] Writer-1 | write #5 | 20:52:51.174
[OWN] Writer-2 | write #5 | 20:52:51.174
=== DONE -- output: output_own.txt ===
0.00user 0.00system 0:00.75elapsed 0%CPU (0avgtext+0avgdata
1648maxresident)k
0inputs+16outputs (0major+96minor)pagefaults 0swaps

```

```
>>> Timing STANDARD SEMAPHORE version...
```

```

time ./std_sem
=== STANDARD SEMAPHORE ===
[STD] Writer-1 | write #1 | 20:52:51.330
[STD] Writer-2 | write #1 | 20:52:51.330
[STD] Reader-1 | read #1 | 20:52:51.380 | "Writer-2 | write #1 |
20:52:51.330"
[STD] Reader-2 | read #1 | 20:52:51.380 | "Writer-2 | write #1 |
20:52:51.330"
[STD] Writer-1 | write #2 | 20:52:51.480
[STD] Writer-2 | write #2 | 20:52:51.480
[STD] Reader-1 | read #2 | 20:52:51.480 | "Writer-2 | write #2 |
20:52:51.480"
[STD] Reader-2 | read #2 | 20:52:51.480 | "Writer-2 | write #2 |
20:52:51.480"
[STD] Reader-1 | read #3 | 20:52:51.580 | "Writer-2 | write #2 |
20:52:51.480"
[STD] Reader-2 | read #3 | 20:52:51.580 | "Writer-2 | write #2 |

```

```

20:52:51.480"
[STD] Writer-1 | write #3 | 20:52:51.630
[STD] Writer-2 | write #3 | 20:52:51.630
[STD] Reader-2 | read #4 | 20:52:51.681 | "Writer-2 | write #3 |
20:52:51.630"
[STD] Reader-1 | read #4 | 20:52:51.681 | "Writer-2 | write #3 |
20:52:51.630"
[STD] Writer-1 | write #4 | 20:52:51.780
[STD] Writer-2 | write #4 | 20:52:51.780
[STD] Reader-2 | read #5 | 20:52:51.781 | "Writer-2 | write #4 |
20:52:51.780"
[STD] Reader-1 | read #5 | 20:52:51.781 | "Writer-2 | write #4 |
20:52:51.780"
[STD] Writer-1 | write #5 | 20:52:51.930
[STD] Writer-2 | write #5 | 20:52:51.930
=== DONE -- output: output_std.txt ===
0.00user 0.00system 0:00.75elapsed 0%CPU (0avgtext+0avgdata
1916maxresident)k
0inputs+16outputs (0major+97minor)pagefaults 0swaps
→ lab-5 git:(main) X

```

Both versions ran in 0.75s. `real` time is dominated by `usleep()` delays, not synchronization overhead. `user` and `sys` are effectively zero for both — the own semaphore's condition variable adds no measurable cost at this scale.

3. Explain any encountered problem.

- Empty file on first read so readers starting before any writer finishes would `fseek` on an empty file. Fixed by checking `ftell() == 0` and returning `"(file is empty)"`.
- Trailing newline on last-line seek so `fseek(SEEK\_END)` lands after the final `\n`, making a naive `fgets` return empty. Fixed by walking backwards past trailing newlines before scanning for the line start.
- Non-deterministic writer order so writers interleave unpredictably across runs. This is expected scheduler behavior, not a bug.